

Atelier 9 - Courbe de charge et preuve de gain

Fiche étudiant • M2 • Jour 5 • Linux Bash • 1 h 15

ShopFlow • RNCP 40166 / contribution à B4C8

But de la séance Construire une courbe de charge reproductible, comparer un avant/après avec un seul changement, puis expliquer où apparaît la saturation sans transformer une variation locale en promesse générale.

Objectif et livrable

La présentation du Jour 5 demande de tester la même consultation avec 1, 5, 10 et 20 clients, de relever le débit terminé, la médiane, le p95 et les erreurs, puis de conserver les sorties brutes, la configuration et l'état de la machine. Le kit applique exactement cette logique avec une charge HTTP fermée.

- Scénario par défaut : relations=n1 comme état initial, puis relations=groupe comme correction du N+1. Les réponses doivent rester fonctionnellement équivalentes.
- Variante autorisée : remplacer ce couple par une optimisation issue des ateliers précédents, à condition de garder le même endpoint, le même jeu, la même durée et un seul changement par série.
- Livrable : courbes p95 et débit, résumé CSV, sorties brutes, protocole, observation du pool/PostgreSQL et conclusion limitée au périmètre réellement testé.

Principe Un résultat neutre ou défavorable reste une preuve utile s'il est correctement mesuré et expliqué. Ne cherchez pas un joli pourcentage, cherchez une conclusion défendable.

1 Préparer le laboratoire et vérifier le contrat

Le kit Jour 5 s'appuie sur l'API ShopFlow déjà utilisée au Jour 4. L'API et PostgreSQL restent sur le même poste que le générateur de charge : ce laboratoire mesure un comportement local, pas une capacité de production.

Démarrage

Terminal 1 - serveur

```
cd optimisationBDD/01_server/api
docker compose up -d --wait
npm start
```

Terminal 2 - outils Jour 5

```
source ../../02_Laboratoire/Jour5/Init_Jour5.sh
j5obs
j5pg
j5stats
```

Conserver les versions de Node, Docker et PostgreSQL, ainsi que la quantité de mémoire disponible. Cela fait partie des conditions de mesure.

```
node --version
docker version
docker compose version
docker compose exec -T postgres psql -U cours -d shopflow -c 'SELECT version();'
```

Contrôle fonctionnel avant benchmark

Avant de comparer des durées, vérifier que les deux chemins renvoient les mêmes vingt commandes et les mêmes lignes. Réutiliser le contrôle de l'atelier 7 ou effectuer deux appels isolés. Si les résultats diffèrent, le benchmark n'est pas comparable.

Arrêt immédiat Si l'API ne répond pas, si le jeu du client 42 n'est plus stable, ou si une autre manipulation modifie les données pendant la série, corrigez d'abord l'environnement. Une courbe propre sur un état incohérent ne prouve rien.

2 Le protocole de mesure

Le script jour5_bench.mjs utilise une charge fermée : chaque client virtuel envoie une nouvelle requête après la fin de la précédente. Le ralentissement du service réduit donc naturellement le rythme d'arrivée. Cette caractéristique doit apparaître dans le compte rendu.

Élément	Valeur du protocole	Pourquoi
Clients	1, 5, 10, 20	Construire une courbe, pas une mesure isolée
Échauffement	10 s par point	Limiter l'effet du premier appel et des caches froids
Mesure	30 s par point	Même fenêtre pour toutes les séries
Modèle	Charge fermée	Un client repart après sa réponse
Timeout HTTP	15 s	Évite une attente infinie et transforme le blocage en erreur visible
Arrêt anticipé	Si le poste sature ou devient instable	Ne pas pousser le PC de laboratoire jusqu'au plantage

Indicateurs à lire ensemble

Indicateur	Lecture
Débit terminé	Requêtes effectivement achevées par seconde.
p50	Médiane : la moitié des requêtes est plus rapide, l'autre moitié plus lente.
p95	Latence de queue : 95 % des requêtes sont en dessous de cette valeur.
Erreurs	Timeout, HTTP non 2xx ou autre échec. Un gain de débit avec davantage d'erreurs n'est pas un gain valide.
SQL/requête	Moyenne de X-SQL-Count. Elle explique le mécanisme N+1, mais ne remplace pas la durée HTTP.

À ne pas mélanger Le temps HTTP inclut routage, sérialisation, pool et réseau local. Une amélioration SQL ne se transforme pas automatiquement en gain HTTP identique.

3 Série AVANT : N+1

Le premier scénario demande vingt commandes avec leurs lignes en mode n1. Une page non vide produit normalement 1 + N requêtes SQL. Lancez toute la courbe en une seule commande.

```
node "$J5_BENCH" run \  
--label avant_n1 \  
--path '/commandes?limit=20&pagination=curseur&relations=n1' \  
--clients 1,5,10,20 --warmup 10 --seconds 30
```

Le dossier resultats reçoit le détail de chaque requête, le résumé par niveau de charge et deux courbes SVG. Conserver ces fichiers tels quels : ils constituent les sorties brutes de la série.

Clients	Débit req/s	p50 ms	p95 ms	Erreurs %	SQL/requête
1	à relever	à relever	à relever	à relever	à relever
5	à relever	à relever	à relever	à relever	à relever
10	à relever	à relever	à relever	à relever	à relever
20	à relever	à relever	à relever	à relever	à relever

Pendant un point de charge

Dans un troisième terminal, relever au moins une fois l'état du pool, des sessions PostgreSQL et de la machine. Le but est de savoir si la latence vient d'une attente, du moteur ou de la ressource locale.

```
j5obs  
j5pg  
j5stats
```

4 Série APRÈS : chargement groupé

Le second scénario change uniquement relations=n1 en relations=groupe. La pagination, le nombre d’objets, le jeu de données et les durées restent identiques. C’est la condition qui permet d’attribuer la différence au mécanisme testé.

```
node "$J5_BENCH" run \
--label apres_groupe \
--path '/commandes?limit=20&pagination=curseur&relations=groupe' \
--clients 1,5,10,20 --warmup 10 --seconds 30

node "$J5_BENCH" compare --before avant_n1 --after apres_groupe
```

La commande compare produit un tableau de réduction du p95 et de facteur de débit, plus deux graphiques avant/après. Les pourcentages ne sont interprétables qu’aux niveaux de charge présents dans les deux séries.

Clients	p95 avant	p95 après	Réduction p95	Débit avant	Débit après	Erreurs
1	...	...	...	...	...	...
5	...	...	...	...	...	...
10	...	...	...	...	...	...
20	...	...	...	...	...	...

Calcul Réduction p95 = (avant - après) / avant. Facteur de débit = après / avant. Ces deux expressions ne disent pas la même chose et ne doivent pas être mélangées.

5 Lire la saturation au lieu de chercher un record

La courbe devient intéressante lorsque le débit cesse d'augmenter proportionnellement tandis que le p95 ou les erreurs progressent. Le "coude" de la courbe est plus informatif qu'un maximum isolé. Comparez toujours des points de même concurrence.

1. Repérer le premier niveau où le p95 s'accélère nettement ou où les erreurs apparaissent.
2. Vérifier si le débit continue d'augmenter, stagne ou baisse.
3. Relier ce comportement à une observation : attente de pool, sessions actives, verrous, CPU, mémoire ou I/O.
4. Comparer le même point avant/après et décrire le mécanisme qui peut expliquer la différence.
5. Noter les effets secondaires : plus de mémoire, plus de connexions, coût d'écriture, cache plus agressif ou complexité supplémentaire.

Option : utiliser votre optimisation principale

Vous pouvez remplacer N+1/groupe par une autre optimisation déjà réalisée : index d'historique, pagination, PgBouncer, cache ou autre variante validée. Le protocole reste identique. Si le changement modifie le résultat fonctionnel, le test n'est plus un avant/après comparable.

Exemple de conclusion acceptable « Sur ce poste, avec ce jeu et cette charge fermée, la correction N+1 réduit le nombre moyen de requêtes SQL par page et abaisse le p95 à 10 clients sans augmenter les erreurs. À 20 clients, la machine locale devient le facteur limitant. »

Conclusion excessive « PostgreSQL est deux fois plus rapide » dépasse le périmètre testé et masque l'API, le pool et la machine de laboratoire.

6 Dossier de preuves Atelier 9

Preuve	À conserver	Ce qu'elle démontre
P9-1 Protocole	Endpoint, jeu, clients, warmup, durée, versions	Comparabilité et reproductibilité
P9-2 Fonctionnel	Réponses équivalentes avant/après	L'optimisation ne change pas le contrat
P9-3 Courbes	p95 et débit avant/après	Comportement selon la concurrence
P9-4 Sorties brutes	*_brut.csv et *_resume.csv	Mesures vérifiables, pas un graphique isolé
P9-5 Observation	Pool, pg_stat_activity, Docker stats	Cause probable ou ressource saturée
P9-6 Conclusion	Gain, neutralité ou régression + limites	Capacité à interpréter sans sur-généraliser

Questions de restitution

- Pourquoi le modèle de charge est-il qualifié de fermé ? Quel biais cela peut-il introduire lorsque le système ralentit ?
- Pourquoi faut-il lire p95, débit et erreurs ensemble ?
- Que prouve X-SQL-Count dans le scénario N+1 ? Que ne prouve-t-il pas ?
- À quel niveau de clients situez-vous le début de saturation et sur quelles observations vous appuyez-vous ?
- Quel test supplémentaire faudrait-il réaliser avant de généraliser la conclusion à un environnement de production ?

B4C8 Cette fiche produit une preuve de gestion optimisée des données dans une API back-end : résultat correct, mécanisme identifié, mesure reproductible et limites expliquées. La preuve alimente le dossier du Bloc 4 mais ne remplace pas la grille certifiante de l'établissement.